

9th Feb 2026 | CS6370

Mid Sem - 9th March
Assignment - get me
deadline

→ Spell-check

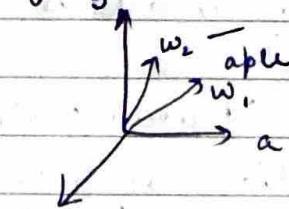
- Top-down
- context-insensitive, sensitive (piece) → more header problem
- words which are not present in dictionary, recovers

Top down → run through dictionary & find similarity (brute force)

Levenshtein distance

(Edit distance Algorithm) → DP

↳ measure how diff. 2 strings are by counting the min. no. of operations needed to change 1 word into another letters as dims. of vector space, words could be the vectors



simple way, but doesn't quite work. We need

sequence. So this is not a effective way.

- ML → Precision - Theoretic ML
↳ Structural / Syntactical ML
- bottom up methods are based on inductive bias (separating +ve & -ve class)
↳ not logically sound
- keys image classificaⁿ

key

Not key

these are skeleton image

- _/_/_
- you can build a grammar for each key.
Then convert it to regular expressions.
- Tree grammar - it describes the rules for building tree structures instead of strings. Normal gram. gives seq. of symbols (string), this generates trees.
 - Graph grammar - defines rules for creating or transforming graphs (instead of trees or strings)

- problem is to recover from the spelling errors
 $\hookrightarrow$ impt. $\rightarrow$ critical is relative ordering of the letters

- letter bigrams $\rightarrow aa - ba - zz$ (26×26)

- collocation $\rightarrow n$ -grams (seq. of n letters/words)

map every word as a vector to this vector space using bigrams

- Levenshtein Coefficient -

- Bigrams are useful, they are efficient than naive vector spaces.